IITH Deep Learning Hackathon 2026

Binary Image Classification using Residual CNN + SE Attention

Vallepu Preethika(AI24BTECH11035)
Gummadidala Geetha Charani(AI24BTECH11013)
Himani Gourishetty(AI24BTECH11011)

May 2026

1 Introduction

This report details the development of a binary image classification system for the IITH Deep Learning 2026 Hackathon. The task requires distinguishing between Class 0 (Negative) and Class 1 (Positive) within 3D-rendered scenes containing simple geometric objects.

Core Objectives & Constraints

Architecture: We developed a custom CNN utilizing Residual Blocks and Squeeze-and-Excitation (SE) attention. In compliance with competition rules, the model was trained entirely from scratch without the use of pre-trained weights.

Scale: The system was trained on 18,000 images and evaluated on a 5,010-image test set.

Goal: Our primary objective was to achieve high classification accuracy while ensuring the model could generalize to the subtle distribution shifts (surface reflectance/shine) present in the test data.

2 Dataset Understanding

Physical Composition & Geometric Logic

The dataset consists of 18,000 training images (224×224 pixels) and 5,010 unlabeled test images. Each image is a synthetic 3D-rendered scene containing a random combination of 3 to 5 geometric objects, namely cubes, cylinders, and spheres, placed against a simple gray gradient background.

Underlying Classification Rule:
The labeling follows a straightforward presence-based rule. Class 0 images contain only cubes and cylinders, while Class 1 images include at least one sphere.

This makes the task fundamentally dependent on identifying whether a spherical object is present in the scene.

Learning Objective:
From a modeling perspective, this becomes a shape recognition problem. While both cylinders and spheres involve curved surfaces, spheres exhibit curvature in all directions, along with distinct shading patterns. The model must learn to capture these geometric and shading cues to make correct predictions.

Complexity Analysis: What makes the task Easy vs. Hard?

Why the task is relatively easy:

- **Controlled Environment:** All images share the same background, lighting, and overall rendering style. This allows the model to focus entirely on object geometry without being distracted by background variations.
- **High Resolution (214×214):** The relatively high resolution helps preserve edge details and smooth gradients, which are important for distinguishing between different shapes.
- **Balanced Classes:** The dataset is evenly split between the two classes, which helps prevent bias during training and supports stable optimization.

Why the task is still challenging:

- **Geometric Ambiguity:** In certain views, cylinders can appear similar to spheres in their 2D projection. This means the model cannot rely on simple shape outlines and must instead learn more detailed shading and curvature information.
- **Shininess (Specularity) Shift:** A subtle but important difference is observed in the test set. Compared to the relatively matte appearance of training images, test objects appear slightly more reflective (shinier).
- **Effect of Specular Highlights:** The increased reflectivity introduces bright highlights on object surfaces. These can slightly distort edges and act as high-frequency noise, making it harder for the model to consistently recognize shapes. This likely contributes to the gap between validation performance and the leaderboard score (76.3%).

Texture vs. Color: An Important Observation

During experimentation, we found that applying ColorJitter negatively impacted performance.

Explanation:
In this dataset, color plays an important role beyond simple appearance. It carries the shading information that helps define the 3D structure of objects. When brightness or color is altered too aggressively, these subtle gradients are

disrupted, making it harder for the model to distinguish between flat and curved surfaces. For this reason, we avoided strong color-based augmentations to preserve these cues.

Train–Validation Performance Behavior

An interesting observation during training was that validation accuracy (often above 99%) exceeded training accuracy.

This can be explained by two factors:

- **Same Data Distribution:** The training and validation sets are split from the same dataset and therefore share identical characteristics. This makes validation data relatively easy for the model to handle once it has learned the training patterns.
- **Effect of Augmentation:** Strong augmentations such as Random Erasing, affine transformations, and resized crops were applied during training to encourage robustness. In contrast, the validation set was kept clean (only resizing and normalization). As a result, validation samples are easier, leading to higher validation accuracy and giving an overly optimistic estimate of performance on the slightly different (shinier) test set.

Data Preprocessing and Augmentation

Data preprocessing plays a vital role in deep learning workflows, as it ensures that input images are standardized and improves the model's ability to generalize to unseen data. To achieve this, the following augmentation techniques were implemented in our code.

Random Resized Crop

This transformation randomly selects a region of the input image and resizes it to a fixed dimension of 128×128, enabling the model to learn scale and positional invariance, thereby allowing it to recognize objects appearing at different sizes.

Random Horizontal Flip

This technique horizontally flips the image with a certain probability. It prevents the model from developing a bias toward specific object orientations and improves robustness.

Random Affine Transformation

Affine transformations introduce controlled variations such as small rotations, translations, and scaling. These transformations simulate changes in object placement and viewpoint, as well as real-world conditions like camera motion and minor distortions, thereby enhancing the model's robustness.

Normalization

The pixel values of the images are normalized using predefined mean and standard deviation values. This step is essential for:

- Faster convergence during training
- Stable gradient updates
- Improved numerical stability

Random Erasing

Random erasing removes a randomly selected region of the image by replacing it with random values. This encourages the model to focus on global contextual features rather than memorizing specific regions, thus reducing overfitting.

Vertical flipping is excluded because it violates the natural orientation distribution of the dataset and does not correspond to plausible real-world transformations.

These augmentations introduce controlled variability in the training data, helping the model learn invariant and robust features.

For the **testing (and validation) data**, only deterministic preprocessing steps are applied:

- Resizing images to a fixed dimension of 128×128
- Conversion to tensor format
- Normalization using the same mean and standard deviation as training

No data augmentation is applied during testing to ensure a consistent and unbiased evaluation of model performance.

Dataset Splitting

The dataset is divided into training and validation subsets using an 85:15 ratio. This split ensures that the model is trained on one portion of the data while its performance is evaluated on unseen samples.

Rationale for Splitting

- **Training Set:** Used for learning the model parameters through backpropagation.
- **Validation Set:** Used to evaluate model performance during training and to guide model selection.

Reproducibility

Reproducibility is a critical aspect of this work and is ensured by systematically controlling all sources of variability in the pipeline.

- **Controlled Randomness:** All stochastic operations, including dataset shuffling and model initialization, are controlled to ensure that the train-validation split and training behavior remain consistent across runs.
- **Deterministic Data Pipeline:** The preprocessing and augmentation pipelines are explicitly defined and consistently applied. While stochastic augmentations are used during training, the validation and test pipelines remain deterministic to ensure fair and stable evaluation.
- **Controlled Experimental Configuration:** Hyperparameters such as learning rate, batch size, and number of epochs are systematically varied during experimentation to study their impact on model performance. However, for each individual experiment, these parameters are kept fixed to ensure consistent and reproducible results. This allows fair comparison between different configurations while maintaining experimental reliability.
- **Consistent Model Selection:** The best model is selected based on validation accuracy using a fixed evaluation protocol, ensuring that model selection is unbiased and reproducible.
- **Hardware and Framework Consistency:** Training is performed using a consistent hardware setup (NVIDIA T4 GPU) and a fixed deep learning framework configuration, minimizing variability due to computational differences.
- **Outcome:** These measures ensure that the entire pipeline—from data loading to final prediction—is stable, repeatable, and yields consistent results across multiple runs.

Finally, separate `DataLoader` instances are created:

- Training loader uses shuffling to improve stochastic learning.
- Validation loader does not use shuffling to ensure consistent evaluation.

Model Architecture (FastCNN)

All components of the pipeline, including model definition, training, and testing, are encapsulated within the `generate_predictions` function to ensure a structured and modular implementation.The proposed model is a custom-designed Convolutional Neural Network (CNN) optimized for both performance and computational efficiency. It integrates convolutional feature extraction, residual

learning, channel attention, and regularization techniques to achieve robust image classification.

The architecture consists of:

- Convolutional layers for feature extraction
- Residual blocks for stable deep learning
- Squeeze-and-Excitation (SE) blocks for attention
- Pooling layers for dimensionality reduction

Convolutional Layers

Convolutional layers are responsible for extracting spatial features such as edges, textures, and shapes.

Layer-wise Architecture Overview

To better understand the flow of information, the layer-wise structure of the model is shown below:

$$
\begin{array}{c}
128 \times 128 \times 3 \\
\downarrow \\
\text{Conv}(3 \to 32) + \text{BN} + \text{ReLU} \\
\downarrow \\
\text{Conv}(32 \to 64) + \text{BN} + \text{ReLU} \\
\downarrow \\
\text{ResBlock}(64) \\
\downarrow \\
\text{MaxPool}(2 \times 2) \\
\downarrow \\
\text{Conv}(64 \to 128) + \text{BN} + \text{ReLU} \\
\downarrow \\
\text{ResBlock}(128) \\
\downarrow \\
\text{MaxPool}(2 \times 2) \\
\downarrow \\
\text{Conv}(128 \to 256) + \text{BN} + \text{ReLU} \\
\downarrow \\
\text{ResBlock}(256) \\
\downarrow \\
\text{SEBlock}(256) \\
\downarrow \\
\text{AdaptiveAvgPool}(1 \times 1) \\
\downarrow \\
\text{Flatten} \\
\downarrow \\
\text{Dropout}(p = 0.2) \\
\downarrow \\
\text{FC}(256 \to 2)
\end{array}
$$

Each convolutional layer is followed by:

- **Batch Normalization**, which stabilizes training and improves convergence
- **ReLU activation**, which introduces non-linearity

Residual Blocks

Residual blocks introduce skip connections, where the input is added to the output of a sequence of convolutional layers. The structure of the block is as follows:

$$\begin{array}{c} \text{Input } (C, H, W) \\ \downarrow \\ \text{Conv}(C \to C,\ 3 \times 3,\ \text{padding} = 1) \\ \downarrow \\ \text{BatchNorm} + \text{ReLU} \\ \downarrow \\ \text{Conv}(C \to C,\ 3 \times 3,\ \text{padding} = 1) \\ \downarrow \\ \text{BatchNorm} \\ \downarrow \\ \text{Addition with input (skip connection)} \\ \downarrow \\ \text{ReLU} \end{array}$$

The output of the block is given by:

$$y = \text{ReLU}(F(x) + x)$$

Parameters Used:

- Number of channels: $C \in \{64, 128, 256\}$ (depending on stage)
- Kernel size: 3×3
- Padding: 1 (to preserve spatial dimensions)
- Bias: Disabled (used with Batch Normalization)

Purpose:

- Mitigate the vanishing gradient problem by enabling direct gradient flow
- Allow deeper networks to be trained effectively

Instead of learning a direct mapping, the network learns a residual function $F(x)$, which improves optimization and stabilizes training.

Squeeze-and-Excitation (SE) Block

The SE block acts as a channel-wise attention mechanism that adaptively recalibrates feature maps.

Working:

- **Squeeze:** Global spatial information is captured using adaptive average pooling, reducing each channel to a single value.
- **Excitation:** The pooled features are passed through two fully connected layers with a bottleneck (reduction ratio $r = 16$) to learn channel-wise dependencies.
- **Scaling:** A sigmoid activation produces weights in the range $[0, 1]$, which are applied to the original feature maps through channel-wise multiplication.

Mathematically, the output is given by:

$$y = x \cdot \sigma(F(x))$$

Parameters Used:

- Channels: $C = 256$
- Reduction ratio: $r = 16$
- Pooling: Adaptive Average Pooling (1×1)

Benefit:

- Enhances important feature channels
- Suppresses less relevant features
- Improves feature representation with minimal computational overhead

Pooling Layers

Max Pooling:

- Reduces spatial dimensions of feature maps (e.g., $128 \times 128 \rightarrow 64 \times 64$)
- Decreases computational cost in deeper layers
- Retains the most dominant features by selecting maximum activations

Max pooling is used in intermediate stages to progressively downsample feature maps while preserving strong feature responses such as edges and object boundaries. This helps the model focus on the most informative spatial patterns and improves computational efficiency.

Adaptive Average Pooling:

- Converts feature maps to a fixed size of 1×1
- Produces a compact representation of each channel
- Reduces the number of parameters before the fully connected layer

Adaptive average pooling is used at the final stage to aggregate global spatial information into a fixed-size representation, independent of input dimensions. This eliminates the need for large fully connected layers and ensures parameter efficiency while preserving essential semantic information.

Dropout

A dropout layer with probability $p = 0.2$ is applied after feature extraction and before the final fully connected layer.

Working: During training, dropout randomly deactivates a fraction of neurons (20% in this case), preventing the network from relying too heavily on specific activations.

Purpose: Dropout reduces overfitting by preventing the co-adaptation of neurons. It encourages the network to learn more robust and distributed feature representations.

Dropout is applied after the feature extraction stage (post Adaptive Average Pooling) where the feature vector is compact (size 256). This helps regularize the final classification layer without affecting earlier spatial feature learning.

Inference Behavior: During evaluation, dropout is automatically disabled, and all neurons are used, ensuring stable and deterministic predictions.

Fully Connected Layer

The final feature vector is passed through a fully connected layer:

- Input: 256 features
- Output: 2 classes

This layer produces the final class scores.

Loss Function

The model is trained using **CrossEntropyLoss** with label smoothing.

Label Smoothing: Instead of assigning full probability (1) to the correct class, a small portion of probability (0.1) is distributed to other classes.

Benefits:

- Prevents overconfident predictions
- Improves generalization

Optimizer and Learning Rate Scheduling

The model is trained using the **AdamW** optimizer along with a **OneCycle Learning Rate Scheduler**.

AdamW Optimizer:

- Uses adaptive learning rates for each parameter
- Applies weight decay (10^{-4}) for regularization

Advantages:

- Faster and more stable convergence
- Better generalization compared to standard Adam due to decoupled weight decay

Learning Rate Scheduling (OneCycleLR):

- The learning rate is initially increased up to a maximum value (10^{-3}), and then gradually decreased during training
- Helps the model escape poor local minima in early stages and fine-tune weights later

Benefit:

- Improves convergence speed
- Leads to better final performance

3 Training Execution and Validation Strategy

While the architecture defines the model's capacity, the training process determines how effectively the model learns from data. Our approach focused on efficient GPU utilization and a strict **best-state selection strategy** to ensure strong generalization.

Training Iterative Loop

The model was trained for 8 epochs using a structured **iterative loop**, where each epoch consisted of repeated updates over mini-batches.

- **Mini-Batch Processing:** A batch size of 32 was used. This allowed more frequent gradient updates per epoch compared to larger batch sizes, helping the model converge faster on subtle geometric features such as spherical curvature.
- **GPU Acceleration:** At each iteration, images and labels were transferred to the GPU using `.to(DEVICE)`. This enabled efficient parallel processing of 128×128 images through the convolutional, residual, and attention layers.

- **Forward and Backward Pass:** After the forward pass, gradients were reset using `optimizer.zero_grad()` to ensure independence between updates. Backpropagation was then performed using `loss.backward()`, followed by parameter updates using `optimizer.step()` with the AdamW optimizer.
- **Scheduled Optimization:** A `OneCycleLR` scheduler was applied and updated at every batch step. The learning rate increased during the early stages (exploration) and decreased during later stages (fine-tuning), enabling stable convergence.
- **Training Metrics:** Training loss and accuracy are tracked for each epoch by accumulating predictions over all batches.

Validation Strategy

To evaluate generalization performance, 15% of the dataset (approximately 2,700 images) was reserved as a validation set.

- **Separation of Concerns:** Unlike the training data, the validation set was kept clean and free from heavy augmentations. This ensured that validation accuracy reflected the model's true ability to recognize geometric structures under standard conditions.
- **Evaluation Mode:** At the end of each epoch, the model was switched to evaluation mode using `model.eval()`. This ensured that dropout and batch normalization behaved deterministically, providing a stable and consistent estimate of performance.

Optimal Model Selection (Epoch 6 Peak)

Rather than selecting the final model, we focused on identifying the point of best generalization.

- **Observation:** The highest validation accuracy (approximately 99%+) was achieved at Epoch 6.
- **Overfitting Indicator:** In Epochs 7 and 8, training loss continued to decrease while validation accuracy plateaued, indicating the beginning of overfitting to training-specific characteristics.
- **Final Selection:** Using a best-state selection strategy, the model weights from Epoch 6 were saved and later reloaded for final inference.

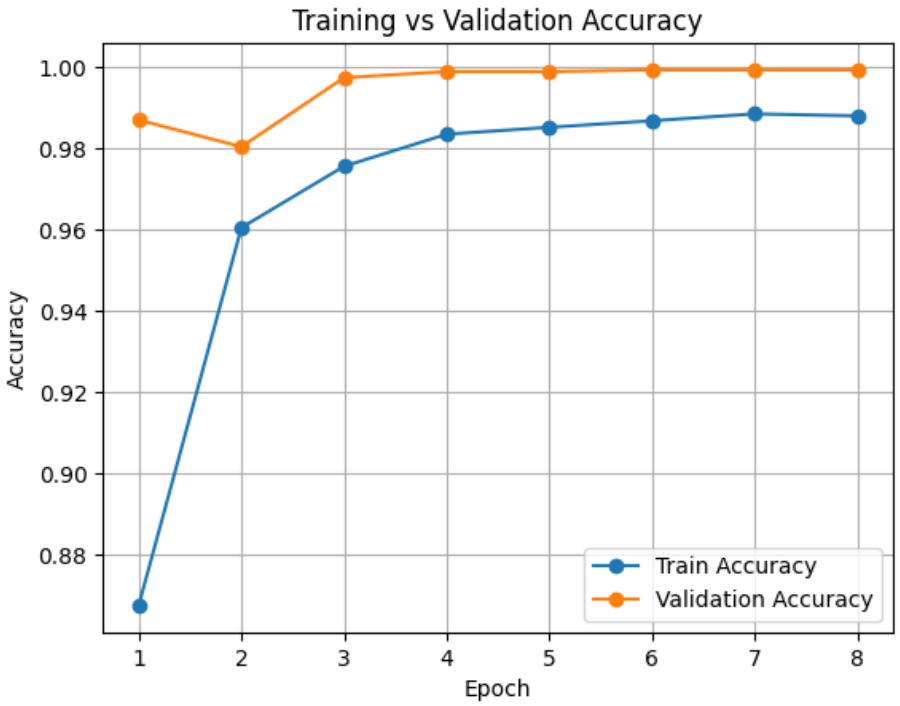
Testing Procedure

The trained model is evaluated on an unlabeled test dataset consisting of 5,010 images.

- **Custom Dataset Loader:** A custom `TestDataset` class is implemented to load test images and return both the image tensor and its corresponding filename.
- **Numerical Sorting:** Test images are sorted numerically based on filenames to ensure correct ordering (e.g., 1, 2, 3, ... instead of 1, 10, 2). This avoids misalignment in the final predictions.
- **Deterministic Preprocessing:** Test images are processed using resizing and normalization only, without any augmentation, ensuring consistent and fair inference.
- **Data Loading:** A `DataLoader` is used with batch size 32 and `shuffle=False` to preserve the order of images.

Inference and Output Generation

- **Evaluation Mode:** The model is set to evaluation mode using `model.eval()` to ensure deterministic behavior.
- **No Gradient Computation:** Inference is performed within a `torch.no_grad()` block to reduce memory usage and improve speed.
- **Prediction:** For each batch, predictions are obtained using the `argmax` operation on model outputs, producing class labels (0 or 1).
- **Result Aggregation:** Predicted labels are paired with their corresponding image filenames and stored in a list.
- **CSV Generation:** The predictions are saved in a CSV file with columns `ID` and `Label`, ensuring compatibility with the required submission format.



4 Approach 1: Transformer-based Model

For our first attempt , we explored transformer-based architectures for image classification. In particular, we took inspiration from the Vision Transformer (ViT) model, wherein the image is segmented into patches. Each patch is linearly embedded into a feature vector, combined with positional embeddings, and passed through multiple transformer encoder layers consisting of self-attention and feed-forward networks. A classification token is used to aggregate global information and produce the final prediction.

Observations

While transformer models are powerful, we observed that they require careful tuning and longer training time to converge effectively. The absence of strong inductive biases, such as locality and translation invariance, makes them more dependent on large datasets and extended training.

Under the constraints of limited training time and no use of pre-trained models, training a transformer from scratch proved inefficient. The model showed slower convergence and required significantly more computational resources compared to CNN-based approaches.

Conclusion and Decision

Due to these limitations, we decided not to pursue transformer-based models further. Instead, we shifted our focus to convolutional neural networks, which inherently capture local spatial patterns and are better suited for moderate-sized datasets.

CNNs benefit from strong inductive biases, such as locality and translation invariance, which allow them to learn meaningful features efficiently even with limited data. This resulted in faster convergence, more stable training, and better performance within the given constraints.

5 Approach 2: Baseline CNN and Progressive Improvements

After exploring transformer-based architectures, we shifted our focus to convolutional neural networks (CNNs) as a more practical solution under the given constraints.

Model Design

We began with a standard CNN architecture consisting of multiple convolutional blocks. Each block included:

- Convolutional layers for feature extraction
- Batch Normalization for stable training
- ReLU activation for non-linearity
- MaxPooling for spatial downsampling

As the network depth increased, the number of channels was progressively expanded (e.g., $32 \rightarrow 64 \rightarrow 128$) to capture more complex features. Finally, global feature aggregation was performed using Adaptive Average Pooling, followed by fully connected layers for classification.

Experiments and Improvements

We conducted several experiments to improve the baseline model:

- **Data augmentation:** Horizontal/vertical flips, rotations, and color jitter were tested. However, strong color-based augmentations were found to degrade performance, indicating that color was an important feature.
- **Image resolution:** Smaller image sizes (e.g., 64×64) enabled faster training but reduced accuracy. Increasing resolution improved feature representation.

- **Regularization:** Dropout layers were introduced to reduce overfitting and improve generalization.
- **Optimization:** Adam/AdamW optimizers with learning rate scheduling (OneCycleLR) improved convergence speed and stability.

Observations

The baseline CNN achieved a test accuracy of approximately 72%, demonstrating that convolutional architectures are effective for this task. However, despite extensive tuning, performance improvements plateaued, indicating limitations in representational capacity.

Conclusion

These observations motivated the transition to more advanced architectures incorporating residual connections and attention mechanisms, which ultimately led to better performance.

6 Approach 3: Efficient CNN with Depthwise Convolutions

To improve computational efficiency, we explored a lightweight CNN architecture inspired by MobileNet, using depthwise separable convolutions. These layers reduce computational cost by decomposing standard convolution into depthwise and pointwise operations.

Model Design

The architecture consisted of stacked depthwise convolution blocks with batch normalization and non-linear activations. Residual connections were incorporated in certain blocks to improve gradient flow and training stability.

Training Enhancements

We introduced several advanced training techniques:

- Mixup augmentation to improve generalization
- Label smoothing to reduce overconfidence
- Test-Time Augmentation (TTA) for more robust predictions
- Learning rate scheduling using OneCycleLR

Observations

Despite the theoretically strong design, this approach achieved a test accuracy of approximately 50%, which is close to random guessing for a binary classification task.

We observed that the combination of multiple regularization techniques led to **over-regularization**, preventing the model from effectively learning meaningful patterns from the data.

Conclusion

Due to this underfitting behavior, we concluded that the model complexity and training strategy were not well aligned. This motivated a shift towards residual-based CNN architectures with more balanced regularization, which provided better learning capacity and improved performance.

7 Approach 4: Residual CNN with Channel Attention

Ablation Studies

Due to time constraints, we did not perform fully isolated ablations for each component. Instead, we experimented with different configurations of the model and observed their relative impact on validation performance.

The ablations presented below reflect qualitative observations based on these experiments, where multiple components were sometimes modified together. While not strictly controlled, these experiments still provide useful insights into the contribution of key design choices.

7.1 Effect of Model Complexity

We experimented with a deeper ResNet-34 architecture with SE blocks and higher input resolution. However, this configuration achieved a lower test accuracy of approximately 75.33% compared to our final ResNet-18 based model, which achieved 76.33%.

We observed that training and validation accuracies were closely aligned, indicating that overfitting on the training data was not a major issue. However, the test set appeared to differ slightly in distribution from the training data, leading to a drop in performance.

The deeper model likely introduced optimization challenges and unnecessary model capacity for this dataset.This suggests that a moderately sized architecture such as ResNet-18 is better suited for this task, providing a good balance between representation capacity and generalization.

7.2 Effect of Handcrafted Feature Augmentation

We experimented with augmenting the input by adding an additional channel representing local contrast (designed to capture specular highlights). This resulted in a 4-channel input instead of the standard RGB input.

However, this modification did not lead to an improvement in performance. We observed that introducing this handcrafted feature increased model complexity and led to inconsistencies between training and evaluation, as the additional channel was applied only during validation and testing.

Furthermore, the standard ResNet-based model using only RGB inputs was able to learn sufficient feature representations directly from the data, making the handcrafted feature unnecessary. This suggests that, for this task, learned features are more effective than manually engineered ones.

7.3 Effect of Image Resolution and Data Split

We experimented with increasing the input resolution to 160×160, using a larger validation split (80-20), and applying stronger regularization through increased dropout(upto 0.4).

However, these changes did not lead to any noticeable difference in test accuracy compared to our baseline configuration. Despite the higher input resolution and longer training, the model performance remained largely unchanged.

This suggests that the baseline configuration was already sufficient to capture the relevant features in the dataset, and further increasing resolution or modifying the data split did not provide additional benefit.

7.4 Effect of Color-Based Augmentation

We initially applied ColorJitter with brightness = 0.4, contrast = 0.4, saturation = 0.3, and hue = 0.05. This resulted in a decrease in both validation and test accuracy compared to the baseline model without color augmentation.

Since the test set appeared to differ slightly in lighting conditions, we further experimented with milder adjustments, restricting ColorJitter to only brightness and contrast (up to 0.2) and removing saturation and hue changes. However, this also did not improve performance and led to a slight decrease in test accuracy($\sim 75.2\%$).

Across all these settings, the best performance was consistently obtained without any color-based augmentation. This suggests that the model is sensitive to the original color characteristics of the dataset, and that naive augmentation strategies are insufficient to bridge the gap between training and test distributions.

7.5 Effect of Controlled Color and Sharpness Augmentation

To better handle the observed lighting differences, we experimented with more controlled augmentations. Specifically, we applied brightness and contrast adjustments up to $\pm 20\%$ using ColorJitter with probability 0.3, instead of applying it uniformly. Additionally, we introduced RandomAdjustSharpness with a factor of 2 (probability 0.3) to capture specular reflections and texture variations.

We also increased the number of training epochs from 8–12 to 25 to allow better adaptation to these augmentations.

This approach resulted in a marginal improvement over strong color augmentation (test accuracy $\sim 75.3\%$), but still underperformed compared to the baseline without color-based augmentation.

Overall, these results suggest that even carefully controlled augmentations are insufficient to address the distribution shift, and preserving the original color characteristics of the data remains the most effective strategy.

7.6 Effect of Threshold Tuning

We applied threshold tuning on validation predictions by selecting a classification threshold that maximized validation accuracy, instead of using the default threshold of 0.5.

However, this modification did not improve generalization to the test set. In fact, test accuracy decreased to 75.21%, indicating that the threshold optimized on validation data did not transfer effectively to the test distribution.

7.7 Effect of Learning Rate and Gaussian Blur

We experimented with modifying the learning rate and introducing Gaussian blur augmentation. Specifically, the learning rate was reduced from 1×10^{-3} to 7×10^{-4} to enable more stable optimization.

In addition, we introduced Gaussian blur with kernel size 3 and probability 0.2 to account for differences in brightness and specular reflections between the training and test sets.

This configuration achieved a test accuracy of approximately 75.2%, compared to 76.3% obtained by the baseline model without Gaussian blur. This indicates that, although Gaussian blur was intended to improve robustness to lighting variations, it led to a slight decrease in performance.

Overall, these results suggest that smoothing-based augmentations such as Gaussian blur may remove important fine-grained details required for classification, outweighing their potential benefits.

7.8 Effect of Mixup Augmentation

We introduced mixup augmentation with a probability of 0.3 during the initial training epochs, where input images and their corresponding labels are linearly

combined.

However, this resulted in a decrease in test accuracy compared to the baseline model without mixup (76.3%). This suggests that mixup negatively impacts performance for this task.

We attribute this to the fact that the classification relies on clear visual features, and mixing images introduces ambiguity in both inputs and labels, making it harder for the model to learn precise decision boundaries.

Overall, these results indicate that mixup is not suitable for this dataset and that training on clean, unaltered labels yields better performance.

7.9 Effect of SE Block

Initially, our model did not include SE (Squeeze-and-Excitation) blocks. We later introduced SE blocks in the deeper layers to enable channel-wise feature recalibration.

We observed that adding SE blocks led to an improvement in test accuracy compared to the baseline model without SE. This indicates that channel attention helps the model focus on more informative features, improving overall performance.

These results demonstrate that incorporating SE blocks enhances feature representation and contributes positively to generalization.

7.10 Effect of Image Resolution

We evaluated the impact of input image resolution on model performance. Training with a lower resolution (e.g., 64×64) resulted in reduced validation accuracy, indicating that important fine-grained details were lost.

Increasing the resolution to 128×128 improved performance, suggesting that this resolution is sufficient to capture the necessary discriminative features.

However, further increasing the resolution to 224×224 did not lead to any improvement in test accuracy. This indicates that beyond a certain point, higher resolution does not provide additional useful information and may introduce unnecessary computational complexity.

Overall, these results suggest that 128×128 provides an optimal balance between capturing relevant details and maintaining efficient training.

8 Approach 5: ConvNeXt-style Architecture with SE Blocks

8.1 Model Design

We explored an alternative architecture inspired by ConvNeXt, incorporating depthwise separable convolutions and modern residual block design. Each block consists of a depthwise convolution followed by pointwise convolutions with GELU activation.

Table 1: Ablation Study on Model Components

Component / Change	**Configuration**	**Test Accuracy**
Model Depth	ResNet-34 + SE	75.33%
Baseline Model	ResNet-18 + SE	**76.33%**
Handcrafted Feature	+ Contrast Channel (4-channel input)	No improvement
Image Resolution	$64 \rightarrow 128 \rightarrow 224$	Best at 128
Color Augmentation	Strong ColorJitter	$\sim$75.2%
Controlled Augmentation	Mild Color + Sharpness	$\sim$75.3%
Threshold Tuning	Optimized threshold on validation	75.21%
Gaussian Blur + LR Change	Blur (k=3), LR 7×10^{-4}	$\sim$75.2%
Mixup Augmentation	Mixup (p=0.3)	$\sim$75.33%
SE Block	Added channel attention	Improved performance

The model is organized into multiple stages with increasing channel dimensions ($32 \rightarrow 64 \rightarrow 128$), and SE blocks are included in deeper layers to enhance channel-wise feature representation.

8.2 Training Enhancements

The model was trained using AdamW optimizer with OneCycleLR scheduling. Standard geometric augmentations such as random crops, flips, and affine transformations were applied, while color-based augmentations were avoided based on prior observations.

8.3 Observations

Despite the more advanced architectural design, this approach did not improve performance compared to our baseline CNN. The model achieved lower test accuracy indicating that the increased complexity did not translate to better generalization.

8.4 Conclusion

We observed that the ConvNeXt-style architecture achieved a test accuracy of approximately 75%, which is comparable to our baseline model. The difference in performance was marginal (within 1%) and falls within expected variation across runs.

Despite its increased architectural complexity, the ConvNeXt-style model did not provide a consistent or meaningful improvement in performance. In contrast, our residual CNN with SE attention achieved similar or better results with lower optimization complexity.

This suggests that increasing architectural sophistication does not necessarily lead to better performance for this dataset, and that a well-balanced model is more effective than a more complex alternative. Through experimentation,

Table 2: Comparison of Different Approaches

Approach	**Test Accuarcy**
Transformer (ViT)	–
Baseline CNN	0.72
Depthwise CNN (MobileNet-style)	0.50
Residual CNN + SE (ResNet-18)	0.763
ConvNeXt-style + SE	0.75

the following observations were made:

- CNN-based models performed better than transformer-based approaches under limited data and training constraints.
- Color information plays an important role in capturing 3D structure, and strong color augmentation negatively affected performance.
- Moderate model complexity provided a good balance between learning capacity and generalization.

Overall, the final architecture was able to capture the underlying classification rule while maintaining robustness to minor distribution shifts.

9 Limitations

Despite the performance gains achieved with our final Residual CNN + SE architecture, several key limitations were identified through our iterative experimentation process:

9.1 Sensitivity to Distribution Shift (The "Shine" Gap)

Observation: While we achieved near-perfect accuracy on validation data, the test accuracy plateaued at 76.3%.

Cause: The test set exhibits a distribution shift in surface reflectance (specular highlights). Our ablation studies (Section 7.5, 7.7) indicate that neither Gaussian blur nor controlled color augmentations effectively bridge this gap, suggesting partial overfitting to the training distribution.

9.2 Constraints of Computational Complexity

Observation: Increasing model depth (ResNet-34) led to a decrease in test accuracy (75.33%).

Cause: Under a constrained training setup (limited time and training from scratch), deeper models introduce optimization challenges while providing limited performance gains. The dataset size (18,000 images) is insufficient to fully exploit higher-capacity architectures.

9.3 Failure of Global Attention Mechanisms

Observation: Vision Transformers (ViT) failed to converge within the allotted time.

Cause: The task relies primarily on fine-grained local features (e.g., shading gradients), rather than global relationships. Without the inductive bias of locality present in CNNs, transformer-based models struggle to learn effective representations in this setting.

9.4 Ineffectiveness of Advanced Regularization

Observation: Mixup reduced performance, and depthwise CNNs achieved only 50% accuracy.

Cause: Mixup introduces label ambiguity that affects precise decision boundaries. Additionally, depthwise separable convolutions reduce representational capacity, limiting the model's ability to capture subtle 3D lighting cues.

9.5 Resolution Saturation

Observation: Increasing resolution from 64 to 128 improved performance, but further increasing to 224×224 yielded no additional gains.

Cause: The discriminative features (shading and curvature) are sufficiently captured at moderate resolutions. Higher resolutions introduce computational overhead without improving useful signal.

10 Future Work

The following directions may improve performance further:

- Domain adaptation techniques to address distribution shift between training and test data
- Improved augmentation strategies that preserve shading information
- Ensemble methods combining multiple models
- Longer training with refined learning rate schedules
- Pretraining on related datasets (if permitted)

Additionally, hybrid architectures combining CNNs and transformer-based attention mechanisms could be explored.

Team Member Contributions

- **Member 1 (Himani):** Himani contributed to the design and implementation of the CNN architecture, including the integration of residual

blocks and Squeeze-and-Excitation (SE) components. She actively participated in model training, experimentation, and hyperparameter tuning to improve performance. Additionally, she assisted in writing the sections related to experiments, ensuring that the methodology and results were clearly presented.

- **Member 2 (Geetha):** Geetha was responsible for data preprocessing and augmentation strategies, designing an effective transformation pipeline to enhance model generalization. She also contributed to debugging issues during training, improving training stability, and optimizing performance. Furthermore, she assisted in documenting dataset understanding, preprocessing techniques, and explaining the model architecture in the report.
- **Member 3 (Preethika):** Preethika explored alternative model architectures, including transformer-based approaches and lightweight CNN variants, to compare performance with the proposed model. She contributed to conducting comparative experiments and analyzing the results. In addition, she assisted in evaluation, test inference, and the overall preparation and refinement of the final report.

11 References

- Course materials from *AI1013: Programming for AI* (CNNs, residual learning, optimization techniques)
- He et al., *Deep Residual Learning for Image Recognition*, 2015
- Hu et al., *Squeeze-and-Excitation Networks*, 2018
- Kingma and Ba, *Adam: A Method for Stochastic Optimization*, 2014
- Smith, *Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates*, 2017
- Assistance from LLMs (ChatGPT) for debugging, structuring, and report refinement